

CS-4110 HPC with GPUs

Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E

Performance Analysis

- Performance tuning is an iterative process
- Requires optimization in:
 - memory access
 - compute efficiency
 - workload balancing
 - communication overhead.
- How to analyze the Performance?
- Proper profiling and understanding hardware limitations are key to achieving peak performance

FLOPS

- Floating point operations per second

FLOPS: Types

- Theoretical FLOPS
 - ~ maximum number of FLOPS a computer can do based on its architecture and specifications
- Measured FLOPS
 - ~ actual computational performance observed during application execution

Theoretical FLOPS

- Measure of theoretical peak performance

$$\text{FLOPS} = \text{sockets} \times \frac{\text{cores}}{\text{socket}} \times \frac{\text{cycles}}{\text{second}} \times \frac{\text{FLOPs}}{\text{cycle}}$$

Theoretical FLOPS

- Servers are the only computers that sometimes have more than one socket; for most home computers (desktop or laptop), “sockets” will be 1.
- Cores per socket depends on your CPU. It could be 2 (dual-core), 3, 4 (quad-core), 6 (hexa-core), or 8. There are some prototype CPUs with as many as 80 cores.
- “Clock cycles per second” refers to the speed of your CPU. Most modern CPUs are rated in gigahertz. So 2 GHz would be 2,000,000,000 clock cycles per second.
- The number of FLOPs per cycle also depends on the CPU. One of the fastest (home computer) CPUs is the Intel Core i7-970, capable of 4 double-precision or 8 single-precision floating-point operations per cycle.

Example

- Intel Core i7-970 has 6 cores. If it is running at 3.46 GHz and can perform 8 floating point operations per cycle, calculate the theoretical compute power of this machine.

Example: Solution

- Intel Core i7-970 has 6 cores. If it is running at 3.46 GHz, the formula would be:
- $1 \text{ (socket)} * 6 \text{ (cores)} * 3,460,000,000 \text{ (cycles per second)} * 8 \text{ (single-precision FLOPs per second)} = 166,080,000,000 \text{ single-precision FLOPs per second}$
- 83,040,000,000 double-precision FLOPs per second.

Theoretical FLOPS: GPU

FLOPS = Cores * Clock * Operations per Core per Cycle

FLOPS = Cores Per SM * Number of SMs * Clock
* Operations per Core per Cycle

Theoretical FLOPS: GPU

For NVIDIA GeForce RTX 3080

Cores: The RTX 3080 has 8704 CUDA cores.

Clock: The boost clock speed of the RTX 3080 is 1.71 GHz (1710 MHz).

Operations per Cycle: Each CUDA core can perform two single-precision (FP32) floating-point operations per clock cycle.

FLOPS per core: $2 \text{ operations/cycle} * 1710 \text{ MHz} = 3420 \text{ MFLOPS}$

Total FLOPS = $3420 \text{ MFLOPS/core} * 8704 \text{ cores}$

$= 29,767,680 \text{ MFLOPS}$

$= 29,767,680 \text{ MFLOPS} / 1,000,000$

$= 29.77 \text{ TFLOPS}$

Measured FLOPS: CPU

```
start = clock();  
    for (long i = 0; i < NUM_OPERATIONS; i++) {  
        // Floating-point multiplication and addition  
        a = a * b + c;  
    }  
end = clock();  
  
// Convert to seconds  
time_spent = (double)(end - start) / CLOCKS_PER_SEC;  
  
// Each iteration performs 2 floating point operations  
flops = (NUM_OPERATIONS * 2) / time_spent;
```

Measured FLOPS: GPU

- How to measure FLOPS in case of GPU?

Measured FLOPS: GPU

```
#define NUM_OPERATIONS 1000000000 // 1 billion operations
#define THREADS_PER_BLOCK 256
#define BLOCKS 256

__global__ void flops_kernel(float *a, float b, float c, long iterations) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    float temp = a[idx];
    for (long i = 0; i < iterations; i++) {
        // Floating-point multiplication and addition
        temp = temp * b + c;
    }
    a[idx] = temp;
}
```

Measured FLOPS: GPU

```
long iterations = NUM_OPERATIONS / (THREADS_PER_BLOCK *  
BLOCKS);
```

```
cudaEventRecord(start);  
flops_kernel<<<BLOCKS, THREADS_PER_BLOCK>>>(d_a, b, c,  
iterations);  
cudaEventRecord(end);  
cudaEventSynchronize(end);
```

```
cudaEventElapsedTime(&milliseconds, start, end);
```

```
double time_spent = milliseconds / 1000.0;  
double flops = (double)(NUM_OPERATIONS * 2) / time_spent;
```

Bandwith Measurement

- Bandwidth in the context of data transfer refers to the rate at which data can be moved between CPU and the GPU over the system's memory bus.
- Measured in gigabytes per second (GB/s)
- Represents the efficiency of data transfer between different memory hierarchies in a computing system.
- **Bandwidth=Data Size (Bytes) / Time (Seconds)**

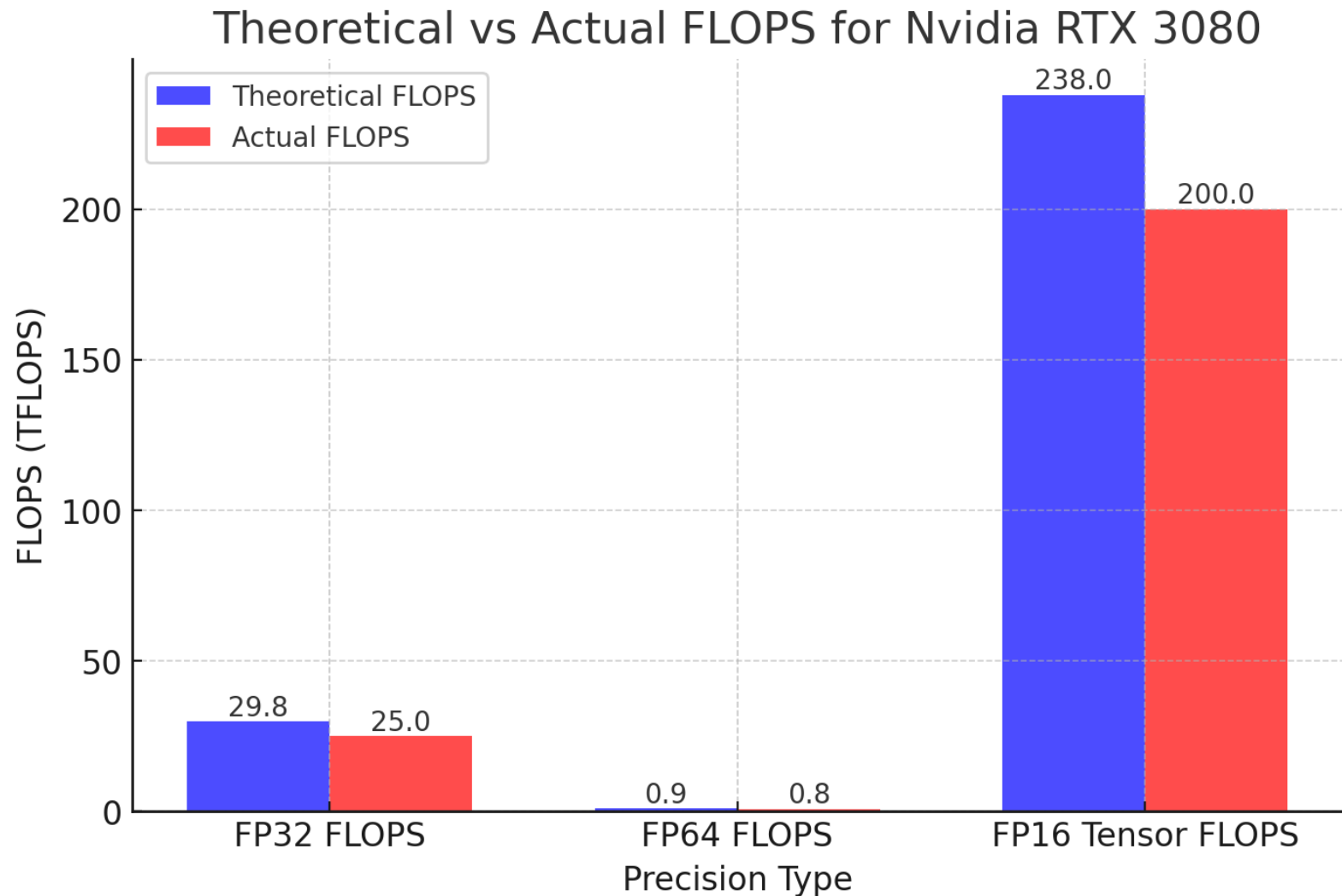
Bandwith Measurement: H2D

```
cudaEventRecord(start, 0);  
cudaMemcpy(d_data, h_data, DATA_SIZE, cudaMemcpyHostToDevice);  
cudaEventRecord(end, 0);  
cudaEventSynchronize(end);  
cudaEventElapsedTime(&elapsedTime, start, end);  
  
double bw_h2d = (double)DATA_SIZE / (elapsedTime / 1000.0)  
                / (1024.0 * 1024.0 * 1024.0);  
  
printf("Host to Device Bandwidth: %f GB/s\n", bw_h2d);
```

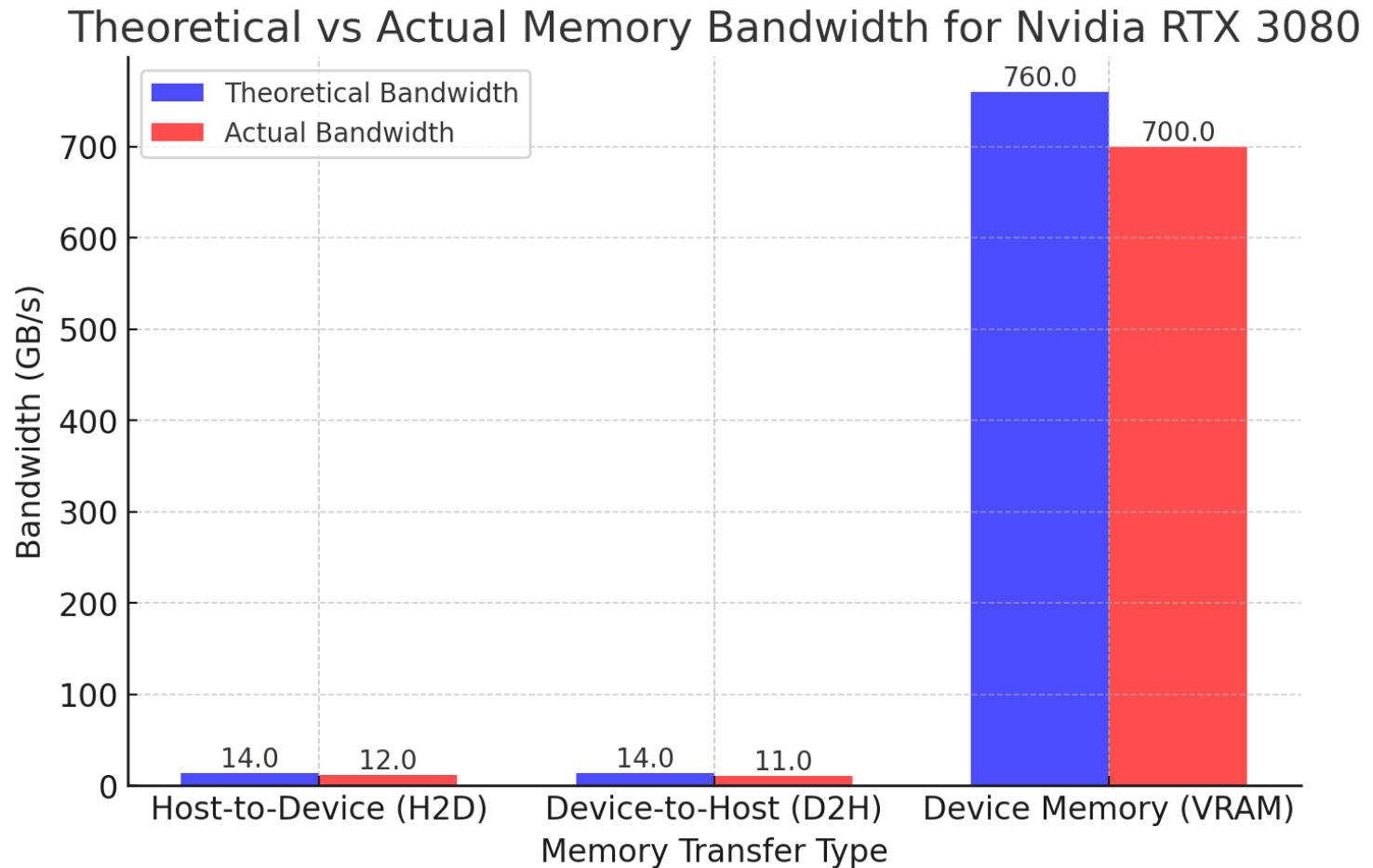
Bandwith Measurement: D2H

```
cudaEventRecord(start, 0);  
cudaMemcpy(h_data, d_data, DATA_SIZE, cudaMemcpyDeviceToHost);  
cudaEventRecord(end, 0);  
cudaEventSynchronize(end);  
cudaEventElapsedTime(&elapsedTime, start, end);  
  
double bw_d2h = (double)DATA_SIZE / (elapsedTime / 1000.0) /  
                (1024.0 * 1024.0 * 1024.0);  
  
printf("Device to Host Bandwidth: %f GB/s\n", bw_d2h);
```

FLOPS: Theoretical vs Actual



Bandwidth: Theoretical vs Actual



Speedup calculations

- Ratio
- %age
- Upper bounds on speedup

Amdahl's Law

- Theoretical speedup calculations
- Formula used to analyze the potential speedup of a program when a portion of it is parallelized
- Provides an upper bound on the performance improvement gained by adding more processing power (e.g., CPU cores, GPUs) to a task.

Amdahl's Law

- Theoretical speedup calculations

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

Example 1

- If 30% of the execution time may be the subject of a speedup, p will be 0.3; if the improvement makes the affected part twice as fast, s will be 2. Amdahl's law states that the overall speedup of applying the improvement will be?

Example 2

- Assume that we are given a serial task which is split into four consecutive parts, whose percentages of execution time are $p_1 = 0.11$, $p_2 = 0.18$, $p_3 = 0.23$, and $p_4 = 0.48$ respectively. Then we are told that the 1st part is not sped up, so $s_1 = 1$, while the 2nd part is sped up 5 times, so $s_2 = 5$, the 3rd part is sped up 20 times, so $s_3 = 20$, and the 4th part is sped up 1.6 times, so $s_4 = 1.6$. By using Amdahl's law, the overall speedup is?

Example 3

- What is better?
- Making 80% of the application 2x faster
- **OR**
- Making 20% of the application 8x faster?

Example 3: illustration

Two independent parts **A** **B**

Original process



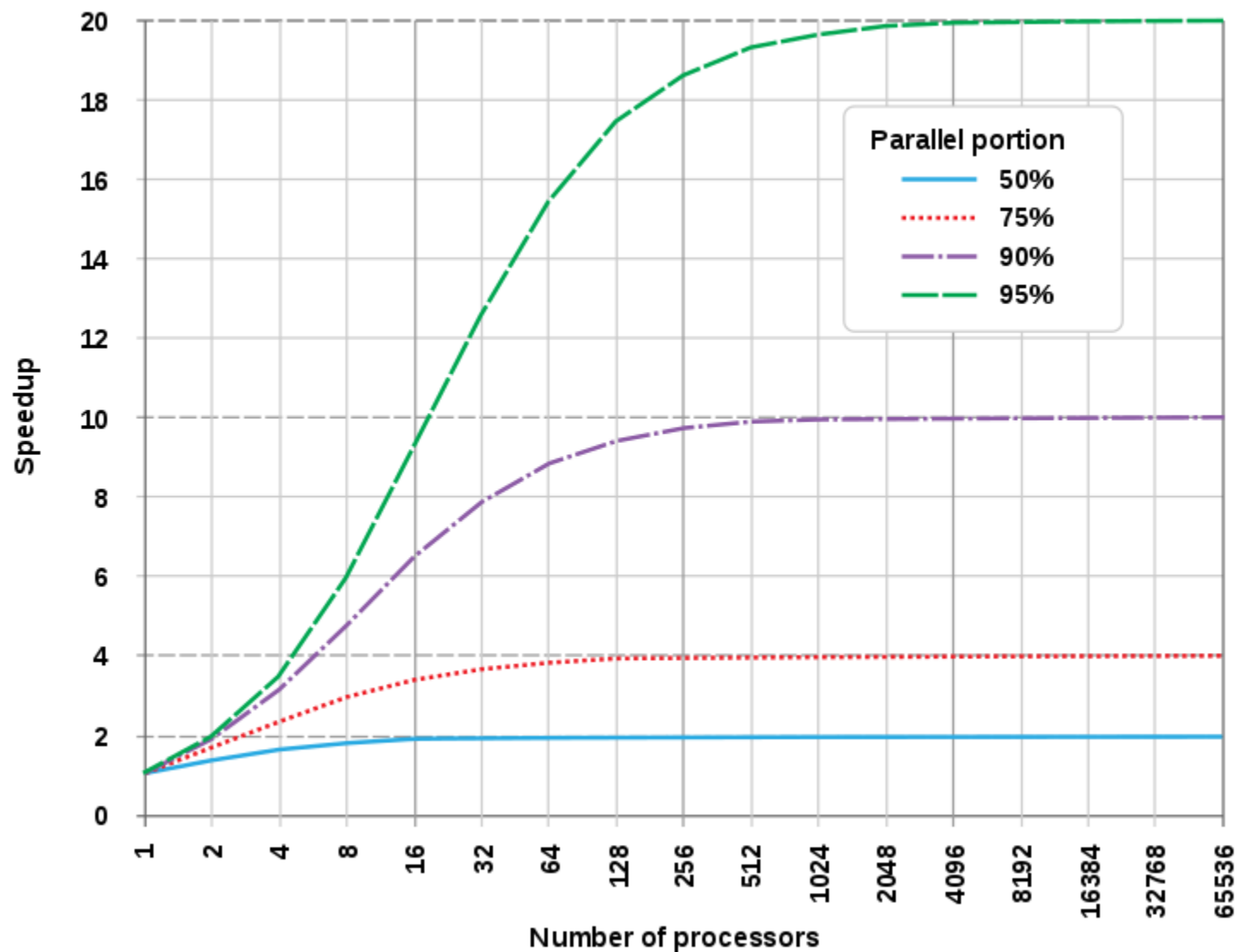
Make **B** 5x faster



Make **A** 2x faster



Amdahl's Law



Amdahl's Law

- Parallelization has limits—even with GPUs, multi-core CPUs, or supercomputers
- Performance improvements are constrained by the portion of the code that must remain serial.
- For best performance, optimize both serial and parallel parts!

Useful Reading

- <https://lwn.net/Articles/250967/>
- <https://people.cs.clemson.edu/~dhouse/courses/405/papers/optimize.pdf>